A Case Study Investigating the Role of Generative AI in Quality Evaluations of Epics in Agile Software Development

Werner Geyer
IBM Research
Cambridge, USA
werner.geyer@us.ibm.com

Jessica He
IBM Research
Bellevue, USA
jessicahe@ibm.com

Daita Sarkar
IBM
Kochi, India
daita.sarkar@ibm.com

Michelle Brachman
IBM Research
Cambridge, USA
michelle.brachman@ibm.com

Chris Hammond*
IBM
Austin, USA
Outrigger Group
Austin, USA
chammond@outrigger.group

Jennifer Heins
IBM
Durham, USA
heinsj@ibm.com

Zahra Ashktorab
IBM Research
Yorktown Heights, USA
Zahra.Ashktorab1@ibm.com

Carlos Rosemberg
IBM
Markham, Canada
carlos.rosemberg@ibm.com

Charlie Hill
IBM
Cambridge, USA
charlesh@us.ibm.com

Abstract

The broad availability of generative AI offers new opportunities to support various work domains, including agile software development. Agile epics are a key artifact for product managers to communicate requirements to stakeholders. However, in practice, they are often poorly defined, leading to churn, delivery delays, and cost overruns. In this industry case study, we investigate opportunities for large language models (LLMs) to evaluate agile epic quality in a global company. Results from a user study with 17 product managers indicate how LLM evaluations could be integrated into their work practices, including perceived values and usage in improving their epics. High levels of satisfaction indicate that agile epics are a new, viable application of AI evaluations. However, our findings also outline challenges, limitations, and adoption barriers that can inform both practitioners and researchers on the integration of such evaluations into future agile work practices.

CCS Concepts

• **Software and its engineering → Agile software development**; • **General and reference → Empirical studies**; • **Human-centered computing → User studies**; • **Computing methodologies → Natural language generation**.

Keywords

Agile Software Development, Agile Epics Quality, Evaluation, Generative AI, Large Language Models, Software Requirements

ACM Reference Format:
Werner Geyer, Jessica He, Daita Sarkar, Michelle Brachman, Chris Hammond, Jennifer Heins, Zahra Ashktorab, Carlos Rosemberg, and Charlie Hill. 2025. A Case Study Investigating the Role of Generative AI in Quality Evaluations of Epics in Agile Software Development. In *CHIWORK '25: Proceedings of the 4th Annual Symposium on Human-Computer Interaction for Work (CHIWORK '25), June 23–25, 2025, Amsterdam, Netherlands.* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/3729176.3729200

1 Introduction

Generative AI continues to be integrated into work practices to support knowledge workers' tasks with the goal of improving the quality and delivery time of their work. Large language models (LLMs) [1] can be effectively used to generate documents, code, or summaries, to analyze or transform data, to translate languages, to answer questions, to classify or categorize content, to brainstorm, or to plan and organize activities [12, 15, 17, 29, 73, 75, 79]. They can also be leveraged to evaluate the quality of both AI-generated and human-created content [20, 26, 42, 80]. For example, AI evaluations have been used in education to assess student work [13, 19, 68], in the legal sector to assess fact relevance [43] and LLM-generated legal texts [58], and in software development [39, 72], where the use of AI has been an active area of research for many years (e.g. [2, 6, 22, 61, 66, 71]). In this work, we investigate the viability of AI evaluations in a new area of work: supporting product managers in creating high quality agile epics. Through this investigation, we contribute to the growing body of research exploring which areas of agile software development would benefit most from generative AI and how to best integrate generative AI assistance into existing workflows (e.g. [3, 6, 18, 41, 45, 50, 53]).

*Now with Outrigger Group.

[1] In the remainder of this paper, we use "generative AI" and "LLMs" interchangeably.

CHIWORK '25, Amsterdam, Netherlands

ACM ISBN 979-8-4007-1384-2/25/06
https://doi.org/10.1145/3729176.3729200

We chose to focus on agile epics as they play a key role in driving agile software development by aligning key stakeholders including product managers, designers, and developers. An agile epic is a representation of a body of work that can be broken down into specific tasks. Epics often describe a problem, a solution, the value to the customer, requirements (often in form of user stories), scope, and target outcomes. They represent a product deliverable that is typically too broad to be completed in a single sprint and hence are broken down into smaller, manageable tasks called user stories [5, 64]. Agile epics are a commonly used tool for product managers to drive work and communicate what needs to be done and why to stakeholders, as described in prior case studies [46, 49, 52]. High quality requirements are critical for the success of software projects [32, 34]. Poorly defined requirements in epics can result in unreliable estimation, delivery delays, churn during development, poor feature quality, and incomplete features – all of which can lead to cost overruns [18].

In this industry case study, we investigate the role that LLMs can play in evaluating agile epic quality as part of the agile software development practices of a large global company. Our goal was to understand how LLM evaluations could be effectively integrated into product managers' work practices, along with their value, usage, and limitations in improving the quality of agile epics. To this end, we conducted a user study with 17 product managers and developed two artifacts to concept test with them: (1) a rubric of quality criteria for epic elements that can be used in both AI and non-AI evaluations, and (2) an interactive, LLM-based tool called Epic Evaluator based on this rubric. Ultimately, we envision that carefully designed tooling based on LLMs will be used and integrated in future work practices to assist agile software development in effectively crafting agile epics to overcome today's challenges of poorly defined requirements. Towards our goal, we address three research questions in this case study:

- RQ1: What are current epic creation work practices and potential implications for LLM-based epic evaluations in agile software development?
- RQ2: What are the characteristics of high quality epics that need to be considered in LLM-based evaluations?
- RQ3: How viable are LLM evaluations in helping product managers understand and improve agile epic quality?

This case study makes the following contributions:

- We introduce and validate a rubric that can be used to assess the quality of an agile epic and outline areas for future improvement of the rubric.
- We introduce a tool called Epic Evaluator, including underlying prompts, that demonstrates how LLMs can be used to evaluate an agile epic.
- We present results from a qualitative user study and concept test with 17 product managers that provide insights into their perspectives on epic quality, opportunities and recommendations for integrating LLM evaluations into work practices, and the value, usage, and limitations of LLM evaluations.

Our findings indicate that LLM-based epic quality evaluations are viable and desired by practitioners but need to provide flexibility to accommodate diverse practices when creating and managing agile epics. At the same time, a tool based on this approach can help organizations encourage standardization with the goal of increasing overall epic quality. We find strong evidence that such a tool would be useful and that there is a clear desire for support in improving epics. However, challenges remain, including the lack of domain and stakeholder knowledge when making recommendations, which affects both the quality and utility of LLM evaluations.

As a case study, our research focuses on agile software development practices in our own company. However, since agile practices are very common and similar across industries, we believe that our insights are valuable, both to researchers who want to advance LLM-assisted decision-making in areas that traditionally require more human involvement, and to practitioners who want to adopt and integrate generative AI in their organizations' future work practices.

2 Background and Related Work

Our paper contributes and builds upon research around software requirement creation and validation in the context of agile software development, AI for software engineering requirements and evaluation, and using large language models for evaluation.

2.1 Agile Software Development

Agile software development is an iterative and adaptive approach to software engineering that emphasizes collaboration, customer feedback, and continuous improvement [70]. Grounded in the principles of the Agile Manifesto [7], this methodology prioritizes individuals and interactions over processes and tools, functional software over extensive documentation, customer collaboration over contract negotiation, and responsiveness to change over rigid planning. While the traditional Waterfall model [36, 57] provides clear structure and predictability, it is less suited for projects with constantly evolving requirements, continuous feedback, and a need for rapid delivery and responsiveness to shifting customer needs. To overcome the limitations of Waterfall models, many companies today have adopted agile practices in their software development lifecycles [11, 25].

While the Agile Manifesto defines foundational principles, agile development practices can vary across different organizations. Agile frameworks such as Scrum [63] and Kanban [1] are often used to structure development into short, incremental cycles, called sprints, to adjust dynamically to evolving requirements. Work is often structured around epics, which capture a larger body of work and break it down into smaller, more manageable units called user stories, which are specific tasks or requirements that can be completed within a sprint [30]. With sprints, the development process flows through short cycles typically lasting two to four weeks, during which cross-functional teams work on user stories. Agile project management software also often supports master epics, which are high-level epics that serve as containers or parent epics for multiple related epics. Compared to traditional requirement documents, epics are not fixed but evolving, do not necessarily specify all system requirements upfront, are written from a user's point of view, are lighter-weight at the documentation level, and are typically managed collaboratively by the product owner and development team. In our own company, the product development process is separated into two phases: discovery, where needs and priorities are defined, and delivery, where the final product is developed

and released. Agile epics (and master epics) can be considered a tool to translate findings from the discovery phase into actionable requirements for the delivery phase of a new product or feature.

Prior work has investigated existing requirements creation and validation processes. In agile development, there is a focus on short iteration cycles, which sometimes leads to insufficient documentation of requirements [35, 47, 67]. For example, Soares et al. [67] identified a lack of detail in user stories as one quality issue. Research around non-functional requirements showed that they are sometimes documented formally such as in epics, but they are also documented throughout other artifacts or not at all [9, 48]. One way to support improved documentation is through guidelines [8]. Some work has begun to provide guidance on improving quality issues. Bik et al. [10] describes a method for writing user stories at a high level, including steps like validating and refining user stories. Ferreira et al. [27] describes a set of guidelines for writing user stories, quality requirements and acceptance criteria, with suggestions like having a "short but suggestive name." Our work contributes to this area of understanding by developing a rubric for agile epics, investigating users' perspectives on the rubric, and demonstrating how an LLM can leverage the rubric for automated evaluations.

2.2 AI for Requirements Creation and Evaluation

Prior to generative AI, tools used traditional AI to support requirements engineering [2, 66]. A survey of practitioners showed that they still rely on prior tools, despite research advancements [71]. Yet, requirements engineering has many opportunities for generative AI augmentation, including new uses that were not previously possible with traditional AI [2, 23, 71].

Research around use of generative AI for requirements engineering has focused on the early stages, like elicitation and analysis of requirements [18, 45]. For example, Oswal et al. [53] created a tool that helps users transform requirements into user stories using GPT-3.5. Dalpiaz and Brinkkemper [22] also developed methods to use AI to create better user stories. A case study evaluating the use of generative AI for writing epics and stories found anecdotal evidence of improved quality, team flow, and speed [14]. They also found that domain knowledge is critical to elicit good outputs from LLMs. A preliminary investigation of the feasibility of using LLMs for requirements elicitation showed that prompts and contextual information were important, as well as experience of the users and domain knowledge [3]. A study of ChatGPT-generated requirements showed that experts often rate them as acceptable, but they can be ambiguous or infeasible [55].

Some research has begun to investigate evaluation of epics and requirements using generative AI. Arora et al. [3] outline the potential for LLMs to support requirements analysis, such as the evaluation of epics. Sami et al. [61] developed a multi-agent approach to requirements engineering using generator agents and a quality control agent. Feedback from four users indicated a potential for this kind of system to support requirements engineering work, but there is little discussion of how the evaluation criteria were created. Wang et al. [76] designed a chat system to support refinement of requirements for software generation, focusing on precision, ambiguity and completeness to improve code generation based on requirements. Lubos et al. [41] use an LLM to evaluate software requirements based on the ISO 29148 standard [28], explain the evaluations, and suggest improvements for the requirements. The LLM was able to identify many quality issues and provide valuable explanations and recommendations. Given the viability of LLM-based evaluations of software requirements demonstrated by prior work, we sought to understand its applicability to agile epics, which capture a higher level view of requirements and contain additional information to drive stakeholder alignment. Our study adds to the body of work demonstrating feasibility of the use of generative AI for requirements evaluation. Additionally, to build on work that found user expertise to be a salient factor in determining usefulness of LLM-support for traditional requirements engineering [3, 55], we further provide qualitative insight into product managers' perspectives on the utility of this kind of tool for agile software development and its potential to be used in their work practices.

2.3 Using Large Language Models for Evaluation

In our work, we leveraged LLMs as evaluators for agile epics. This approach, often called LLM-as-a-Judge, has become very popular in recent years to evaluate the output of other LLMs because it allows developers to flexibly define custom criteria for their use cases without the need for ground truth. However, the same techniques can be used to evaluate text written by humans [20, 26, 42, 44, 65, 80], in domains such as education, the legal sector, or software development [19, 39, 43, 58, 68, 72]. For example, Shaer et al. [65] demonstrate that GPT-4 can be used as an aid to evaluate ideas from brainstorming sessions and Markel et al. [44] outline its use for evaluating students and teachers with GPTeach. LLMs can perform decently well as evaluators of natural language text [26, 80], but there is also room for improvement. In expert knowledge tasks, LLM evaluators and subject matter experts do not always agree [69]. One way to improve performance is to fine-tune models to specifically support evaluation tasks [37]. Further tooling for the evaluator may also help. Tools, like EvaluLLM [24] and Evallm [38], enable custom evaluation of generated text using LLMs. Yet, further assistance is likely needed to support non-experts in authoring effective criteria for these kinds of tools [54]. In our work, we are not leveraging any existing LLM-as-a-Judge tools because most do not support complex, structured definitions of multiple criteria as required by our rubric. Instead, we customized prompts for Epic Evaluator as described in Section 3.2. Our work considers LLM evaluation in the particular domain of agile epics, contributing to the understanding of LLM usage for evaluation of domain-specific natural language text.

3 Methodology

3.1 Rubric Development

To understand existing quality criteria and how they can be applied to LLM evaluations, the first critical step in our investigation was to develop a rubric that can be used to assess agile epics. We started by reviewing top-down guidance on best practices for the software development lifecycle that have been developed and disseminated in our company. The guidelines developed by our agile development

leadership team also include a template for agile epics. We used this template as a starting point to develop our rubric. The template consists of nine elements: (1) title, (2) problem statement, (3) product outcome & instrumentation, (4) user stories, (5) requirements for each user story, (6) assumptions, (7) non-functional requirements, (8) out-of-scope, and (9) questions & decisions. The template provided some examples for how these elements should look like but was sparse on actual quality criteria for each of the elements. For each element, we developed a definition informed by internal and external best practices and quality standards [59, 60, 78], as well as input from LLMs. We used LLMs as an additional source of knowledge and inspiration, as well as an aid to articulate definitions. Based on an initial draft, we iterated with two subject matter experts (SMEs) from the internal agile development leadership team until both were satisfied with the rubric definitions. We excluded "questions & decisions", as suggested by our SMEs, leading to a total of eight elements as shown in Table 1. Detailed definitions and criteria for each element are included in Appendix A.1). As agile epics are broader and somewhat different than traditional requirements specifications, we did not adopt ISO 29148 [28]. However, some of the quality characteristics from the ISO standard are similar to our rubric, such as unambiguous, verifiable, avoiding implementation details, usage of standardized structures, and representing user needs.

3.2 Epic Evaluator: An LLM-Based Evaluation Tool

Based on the rubric, we iteratively developed a number of prompts to evaluate an agile epic. We experimented with various LLMs available[2] to us (llama-3-70b-instruct, llama-3-405b-instruct, mixtral-8x7b-instruct, granite-13b-instruct) and different configurations of our prompts. After various iterations with example epics, we found that llama-3-70B-instruct yields contextually relevant and stable results with reasonable response times when used to assess each element of an agile epic separately rather than one large prompt. In our selection criteria, we defined "contextually relevant" evaluations as those providing examples that demonstrated an understanding of the content of the epic, and "stable" as those that did not change between multiple evaluations of similar epics. While llama-3-405b-instruct also yielded comparable results, response times were significantly higher. We also developed a prompt to identify the presence of each epic element in a text document. This resulted in a total number of nine prompts (see A.3 for an example) that we used to develop a simple Gradio [31] Python application, called Epic Evaluator, which takes the full text of an agile epic as input, analyses the presence of epic elements, and individually evaluates each element found (see Figure 1). Epic Evaluator leverages IBM watsonx.ai as an inference service. For each element, it generates a rating (High/Medium/Low) as defined in A.2, an explanation for the rating, and a recommendation on how to improve the element. While our 3-point rating scale was chosen based on our discussions with SMEs, this simple initial design was mostly intended as a probe for concept testing. The fictitious example shown in Figure 1 illustrates the nature of the explanations and recommendations given.

[2]Access to LLMs in our company for corporate content is limited to a set of approved models which we had to choose from.

Table 1: Elements of our rubric with an abbreviated definition. See Appendix A.1 for the detailed definitions and criteria.

Element	Abbreviated Definition
Title	The title of an agile epic should be short and concise, descriptive, convey the primary objective of the epic.
Problem Statement	The problem statement briefly summarizes the problem which is being solved in this epic including a brief description of the feature developed, and the value for the customer or user. The problem statement should be clear, comprehensive, and concise.
Product Outcome & Instrumentation	This element describes the overall goals of this feature. Product outcomes are what determines this feature's success. Good product goals should measure things like adoption, feature retention, satisfaction, engagement or impressions, general usage, completion percentages and speed. Instrumentation refers to metrics that can be used to measure the goals, e.g. click-throughs etc.
Requirements - User Stories	User stories describe what the user needs or expects from the system. User stories are a fundamental part of an agile epic. User stories identify and break down needs for each user role. They should be clear, structured, and complete.
Requirements - Requirements	The requirements element breaks down each user story into a requirement statement but typically leaves out the reason/benefit found in the user story. It is basically a breakdown of a user story into system capabilities that are required to implement the user story.
Assumptions	Assumptions are the beliefs or statements that the team holds to be true at the start of the epic but which may not yet be fully verified. They often represent things that need to be confirmed or validated during the development process. Assumptions are typically based on what is known at the time but can evolve or be disproved as more information becomes available. They should be explicit, testable, and describe their impact.
Non-Functional Requirements	Non-functional requirements are requirements that are not centered around the user experience. Non-functional requirements should be measurable, testable, realistic, and relevant.
Out-of-Scope	The out-of-scope section clearly states what will not be included in the feature, usually as a list of items. Out-of-scope items should be clear, relevant, specific, and justifiable.

They often, but not always, contain concrete examples on how to improve an element of an epic referencing content within the epic. If an element is not found, it only generates a recommendation. We discussed this design on our team and concluded that this would

be a useful initial approach that we can use for concept testing [33] with our study participants. This design is similar to what Lubos et al. [41] studied quantitatively. In our study, we used Epic Evaluator to evaluate agile epics provided by our study participants.

3.3 User Study

In order to gain insights into practitioners' perspectives on epic quality, opportunities for integration of LLM evaluations into work practices, and the value, usage, and limitations of LLM evaluations, we conducted semi-structured interviews with 17 product managers. During the interviews, we used the rubric and the outputs of the Epic Evaluator tool for concept testing [33] – a technique used to elicit user feedback on ideas or lightweight prototypes before investing large amounts of development resources, which was well suited for the early stages of our artifacts. Seed participants were recruited from groups of product managers who had early exposure to a technology pilot or were recommended by the agile development leadership team. We then used snowball sampling to recruit additional participants. We stopped recruiting new participants once we felt our sample was diverse enough and observations seemed to converge. The 17 participants (9 female / 8 male) were all in product management roles as Product Managers (52.9%), Product Leads (17.6%), and Directors (29.4%). 47.1% of the participants were in a people management role. Their product management experience ranged from 1.5 years to 20 years with an average of 6.6 years.

In our interview protocol, questions were grouped into four categories: (1) background including job role and AI usage, (2) their epic creation process, importance of epic elements, and general quality considerations, (3) concept testing of the rubric we developed including rating scales, frequency of evaluation, and satisfaction, and (4) concept testing of evaluations generated by the Epic Evaluator. During section (2) of the interview, we showed participants a list of epic elements and asked them to rate their importance while thinking aloud. During section (3), we showed them the rubric and asked for feedback for each element of the rubric while thinking aloud. During section (4), we shared screenshots of evaluations of 2-4 elements of an epic they had written, which they provided to us prior to the interview, and asked them to critique the evaluations. For 5/17 participants who were not able to provide data in time for the interview, we prepared evaluations based on internal sample epics. All interviews were conducted by one moderator and one note-taker via video-conferencing and took between 45-60 minutes. Three authors conducted thematic analysis [21] of the interview transcripts by creating an initial code book based on a subset of the data. The code book was subsequently applied to all the transcripts and incrementally extended if any new themes were identified, in agreement with all three analysts. We provide descriptive statistics where appropriate but given the qualitative nature of the study, we do not provide statistical tests. The study followed company policies on user data. Participation was voluntary, with the option to withdraw anytime. Data was de-identified, stored securely, and analyzed anonymously. Participants consented to the use of de-identified data for current and future research, ensuring confidentiality.

4 Results

We present the themes we identified for each research question: implications of current epic creation practices for LLM evaluations, characteristics of high quality epics, and the viability of generative AI in supporting epic evaluation.

4.1 RQ1: What are the current epic creation work practices and potential implications for LLM-based epic evaluations in agile software development?

4.1.1 Overview of epic creation practices. Participants' current approaches to epic creation indicated that it is a highly **collaborative and iterative** process. Although epics are owned by product managers, who typically write the first draft, most participants noted that it evolves with input from stakeholders, which can include other product managers, lead developers, lead designers, software architects, and sometimes marketing teams. Epic creation often evolves through **multiple stages** – as P9 explained, *"the epic goes through different stages of being written: being committed, being in design, then ready for development, in development and ready to ship, and shipped."*

Beyond these similarities, specific **practices diverge** significantly. The starting points, tooling, formats, use of specific elements, creation time, evaluation frequency, and involvement of stakeholders at different stages can all vary. For example, on P5's team, problem identification is owned by product managers, who *"capture it through different feedback sources"*, and *"what we need to do to solve that problem also [..] is product managers' responsibility."* In contrast, P9 approaches problem identification collaboratively: *"we work as three-in-a-[box] and go to hear the customer pain points together, so we can capture from different aspects what they're saying."* The epic evaluation process and timing also vary: as a manager, P7 focuses on the problem statement and outcome to ensure they are aligned with the product focus, while P14 looks across elements to determine whether the right ones are present and clear. To illustrate the high variability in practices, we share two vignettes describing differing processes in Table 2.

Epic creation can vary within teams as well. For example, P8 writes two different types of epics. One is a *"placeholder epic [..] to kinda get it in the system so that you can have something to note for a broader discussion,"* which is held to lower quality standards and only includes a title, basic required fields, and a prioritization report. In contrast, for higher fidelity epics, they start with an *"epic template that we've aligned on as a team"* to ensure *"uniformity"* and *"completeness."*

Another point of variability was in participants' perceived importance of different epic elements. When asked to rate the importance of each epic element from our rubric on a scale of 1-5, with 5 being most important, participants generally agreed that problem statement (M (SD) = 4.87 (0.35)) and user stories (M (SD) = 4.27 (0.86)) were most important, but there was variability in how they weighed the rest of the elements. This variability stemmed from differences in team practices and perceptions of an element's necessity. For

[1]https://www.mural.co/
[2]https://www.aha.io/

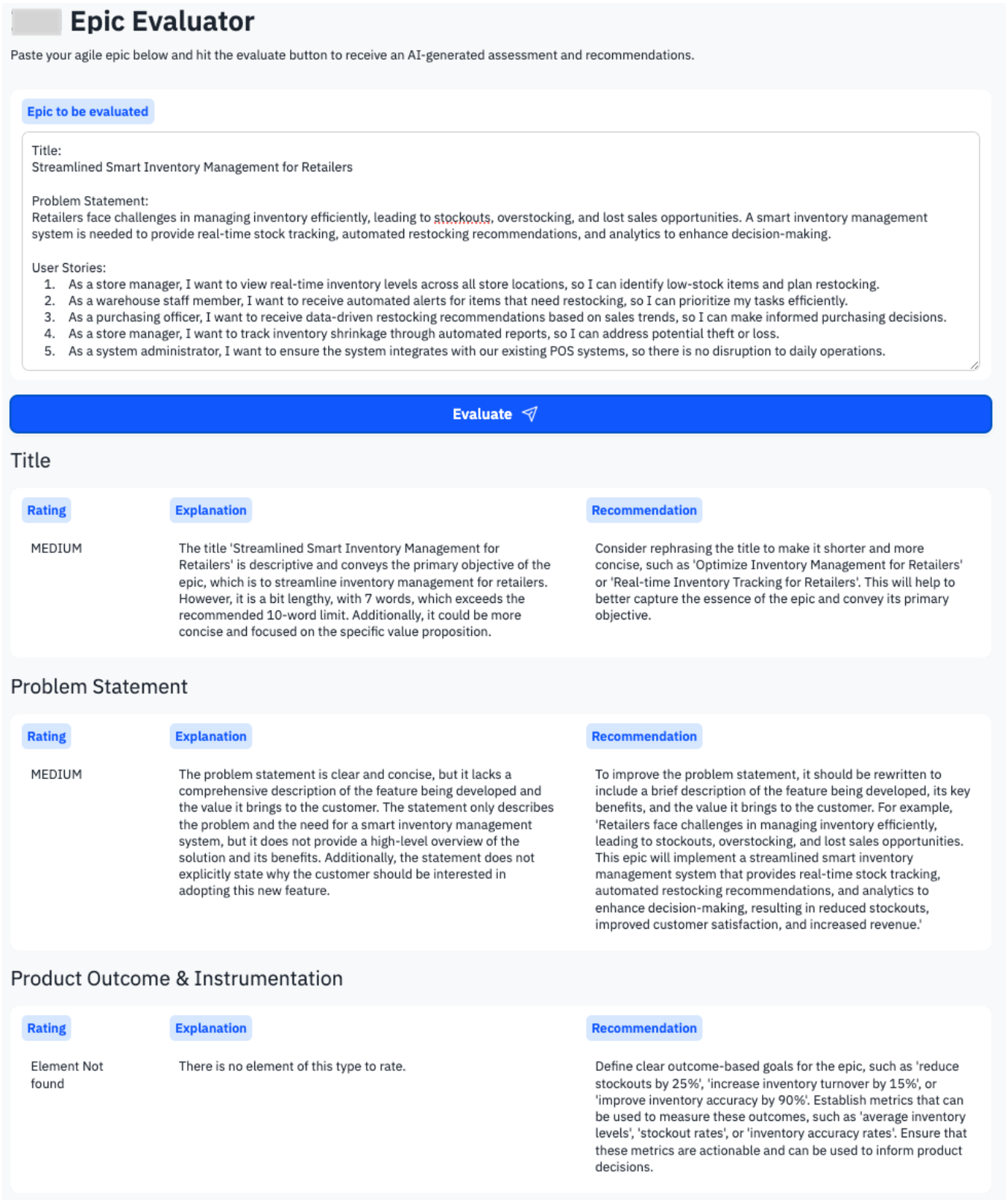

Figure 1: The experimental Epic Evaluator tool. The full text of an agile epic is pasted into the input box at the top. When pressing the 'Evaluate' button, the system checks for the presence of agile epics elements and for those present, rates the quality, provides an explanation, and makes recommendations for improvement.

P14	Product managers mostly only create master epics, which they described as having a larger scope than regular epics. After receiving a request for new functionality, they work with designers and developers to ideate on solutions in Mural [1]. Following ideation, the product manager uses a custom Aha! [2] template with two structured fields: a free-form description where they enter customer personas and links to relevant artifacts (such as ideation boards and prior research), and a value statement that describes who they are solving for and what value they drive from the outcome. P14 views epics as a *"contract with our development team"*. After hand-off, developers then break down the master epic into sub-epics or issues, occasionally coming back to product managers with suggested changes for technical feasibility.
P2	The epic creation process starts with a new technical requirement or customer idea. They use the standard company template to write the first draft of the epic. Once the draft is ready, they work with development leads to scope and define the technical design of the feature. The product manager then writes user stories and acceptance criteria, continuing to involve developers to ensure each story is sprint-sized before handing off for implementation.

Table 2: Vignettes of different epic creation processes

example, P6 felt that product outcome & instrumentation *"is not that important because this lives higher than the epic level, at the team level,"* whereas P9 found outcome very important *"to know where we're taking the product."* Participants rated an element as less important if it is captured outside the epic (e.g., in the product strategy) or considered redundant with another element. They rated elements as more important if considered crucial for organizing, understanding, or executing the work. In some cases, participants considered an element to be theoretically important but not in practice – for example, P13 said, *"I would say outcome is one of the important things, but [product name] is on-prem [..] we do not have any instrumentation for seeing the adoption of the particular feature."*

4.1.2 Opportunities for AI injection. We identified several pain points in participants' current epic creation processes that present opportunities for AI support, particularly through evaluation. AI-based evaluations can help teams **improve specific epic elements** that they want to work on. For example, P8 expressed a desire to improve instrumentation: *"instrumentation, we don't do as [well] as we should, adding this to every epic, but it should be added because if it's not [..] then you lose a lot of the value in the product."* P1 noted, *"we should probably be doing better in terms of performance requirements [..] That is definitely an area, I think, where these requirements generators can help us."* AI evaluations can also help **drive consistency**, which some participants recognized as a need either within their team or across the company. For example, P15, who manages a portfolio of multiple products, recognized that they are *"not entirely consistent across all of [their] products"* in how they scope master epics and expressed a need for *"being a little more consistent across our portfolio in what is in a master epic."* P8 explained the importance of using a consistent format within their team for stakeholder clarity, saying that it *"makes it easier"* for designers and developers when *"everyone is used to it."* At an organization level, P14 explained, *"there's been pockets of people that do things differently, and all their different ways work, but either when teams cross, talk to each other, or people move from team to team [..] there's a lot of inefficiency."* Several participants noted AI could **provide support for product managers who are overwhelmed**. P11, who works on a team in which engineers significantly outnumber product managers, explained, *"I think a lot of teams need help. I'm sure if you go interview engineering colleagues, they'll say PM gives us [under-specified] requirements [..] we need to do better because that's the role of PMs, is actually defining things."* They felt that *"AI can be the accelerator"* in the highly time-consuming epic creation process and spoke to the need for *"interactive and iterative"* support tools.

Beyond the work of individual product managers, another opportunity for LLM support is in stakeholder collaboration. As the epic creation process is highly iterative and collaborative, it is prone to communication and process breakdowns, as mentioned by several participants. P14 illustrates the benefits of good epics and how they can **alleviate communication breakdowns**: *"good epics are a huge factor in efficiency of how we're gonna build and deliver software that [..] if we don't start from good epics [we are] wasting a lot of time in the margins trying to correct bad epics. So I think you're attacking the right problem with the research here. [..] It's probably going to save me five days worth of work in terms of meetings that I have to go into to correct that downstream [..] at a much higher expense."*

Beyond these pain points and opportunities, we also sought to identify *when* LLM evaluation would be most useful in the multi-stage creation process. When we discussed how often an epic should be evaluated, a few mentioned a continuous need, such as P12 who wanted *"to evaluate the same epic over and over again till I know I've got it right."* However, a **pattern of common milestones** emerged as potential points of injection: (1) after completing the first draft, (2) before sharing with stakeholders, (3) after iterations with stakeholders, and (4) before starting development. The above patterns are consistent with the observation that epics are developed in stages and that elements have different levels of importance depending on the stage as P8 points out: *"defining this up front, I think is a mistake. So again, there's a timeline aspect to this and I think that should be factored into the definition."*

4.1.3 Summary. Addressing RQ1, we found that current epic creation practices offer meaningful and valuable opportunities for the injection of LLM evaluations during the creation process such as, reducing communication breakdowns, accelerating epic creation, or simply improving individual elements. As epic creation happens in stages and evaluation is needed at multiple milestones, LLM evaluation will need to take into account the current stage of an epic at the time of evaluation. Given the diversity of current practices, LLM evaluation needs to flexibly address those, but they also offer opportunities for more consistency as desired by many participants. While consistency can be achieved through standardization of what epic elements should be used or configured, standardization also comes in terms of how the quality of each epic element should be defined which brings us to our next research question.

4.2 RQ2: What are the characteristics of high quality epics that matter for LLM-based evaluations?

Understanding what makes an epic high quality is critical for designing LLM-based evaluations but also offers opportunities to standardize on a definition of what defines a "good" epic. In order to validate the assumptions we made when developing the rubric, we first collected participants' perspectives on epic quality without showing them the rubric, then asked them for feedback about our rubric.

4.2.1 Product managers' perspectives on epic quality. When we asked participants about their key considerations when evaluating an epic, we overwhelmingly heard **clarity and understandability** as the most important quality characteristics (9/17). The important role of epics in communicating requirements to design and development was highlighted by P5 who said *"A good epic in my opinion, is something that you know that can get product management, development, design and architecture all clear on what we want to do, and it lists out the problem and the outcome very clearly."* It was also emphasized upon by P10 as, *"I think the most important thing is [..] if someone from the engineering reads that epic and understands what they need to do."*

The second most mentioned characteristic was **completeness** (7/17). For some, this meant that some critical elements of an epic need to be present; for others, it meant all required elements needed to be there. P11 and P17 indicate how the lack of elements can lead to unnecessary churn or indicate communication challenges with development. As P11 explains, *"If you don't have an acceptance criteria or high level user stories statement, that irks me. [..] I'm like, don't talk to development [..] Let's [not] waste our colleagues time [..] if you're not even trying to put some amount of acceptance criteria or any amount of critical thinking, that's when I know that epic wasn't written with any care or concern."* Similarly P17 notes, *"If it's missing [the] trifecta of user stories, requirements and acceptance criteria.[..] just any semblance of that with complete clarity around what the expectation is for what dev's gonna build [..] That's my first sort of like OK, something's not right here."*

In addition to completeness, participants also indicated the importance of **detail and specificity** (6/17). As a measure, some participants indicated that an epic should be understandable to a newcomer or peer. As P14 puts it, *"Does it have all of the right components documented? Is there enough detail in it? Can I understand the story? And you might test that by maybe asking a peer product manager who hasn't been involved in the process if they could read it and they could kind of understand it."* On the other hand, P6 was concerned about excessive content, stating,*"One is definitely how how easy to read it is. I think it's really easy [..] as a PM to make your epics like unnecessarily lengthy and wordy because the the main thing that you're doing is writing the epic. And so you put a lot of detail into it, but then you take it to the devs and designers [..] and it's [..] very hard [..] to digest [the] information."*

Several participants highlighted "semantic" quality characteristics related to the content of an epic, such as meeting customer needs, driving value, or alignment with strategy, which may require contextual knowledge beyond the epic itself and could be more challenging for an LLM to evaluate. As P4 explains, *"One other field that I think is important is the customer value statement [..] Like what is the end value to a customer once we implement this particular functionality?"* P7 adds, *"I'm first looking at alignment [..] so instead of reading the whole [..] thing, I'm gonna look at the problem statement, and then I'm gonna look at the outcome and whether this is aligned to where my focus is [..] I noticed that we were doing a lot of stuff that was not tied to a coherent strategy."*

One manager (P8) offered his own perspective on how he would go about evaluating the quality of epics of his reports: *"So the first thing I'm evaluating as a leader is (1) Is this aligned to what we're focused on and if not, why? [..] (2) What the requirements are, so that I'm now going to the user stories [..] based on the problem that we're trying to solve, like do I actually think that we understand what the user is struggling with and what we're helping them with [..] if that jives, then I'm looking at the completeness of everything else."*

Other quality aspects mentioned less frequently during the interviews were: following structural guidelines, including links to outside documentation, test cases, and dependencies, ease-of-adoption by adjacent services, restricting the epic to the "what" and not "how" it is being implemented, and the need for epic elements that were not part of the standard company template.

4.2.2 Product managers' feedback on the rubric. When we showed participants the rubric, the majority of them expressed their satisfaction (12/17). For instance, P1 said, *"Overall it's very good. It's actually kind of nice to have you guys kind of looking at it this way, [..] more of a scientific view on something that was maybe more of an artwork writing requirements, right."* with some calling it comprehensive, concise, clear, and even scientific, and many were interested in using it as is as a guideline for epic quality.

From their reactions, we had the impression that the level of detail was unexpected but not unwelcome, which was surprising given the diversity of epic creation practices outlined in RQ1. While many were not strictly following the "official" guidelines, participants were mostly familiar with the elements in the rubric, minus some confusion about specific elements (6/17). Most welcomed the clear definitions of how these elements should be written as encouragement for them to strive towards better epics for themselves or their teams. As P14 mentioned, *"I absolutely would use this rubric. I would say in reality today only some of it right. Some of it is intuitive. Yep, that's exactly [what we are] already doing [..]. There's other pieces that I was like, yeah, definitely. I would love to have a check and balance on that because I know it's something that slips through the cracks on me."* P16 humorously added, *"As a user, I would not use any sort of an AI tool to evaluate it after the fact, I would use it to actually write it in the 1st place as a leader."*

As part of the think-aloud exercise when concept-testing the rubric, we identified several pain points with the rubric that describe additional needs when evaluating an epic.

Several participants pointed to **missing elements** required to support their practices. Most notably, the standard template we used for the rubric did not include acceptance criteria for user stories, which many participants (7/17) mentioned to be critical for their epics. In RQ1, we discovered that the problem statement appears to be the most important element for a rubric, and most agreed with the definition of the quality for this element. However,

multiple participants felt the problem statement should only focus on the actual problem, whereas solution and customer value should be broken out as new elements, indicating that the element may be overloaded.

The variability we discovered in RQ1 continued to reflect an important need for participants, as some expressed that the rubric may be too **rigid** for them in terms of how quality is defined. For example, P2 commented on their team's selective use of epic elements: *"everything in this table, like non-functional requirements, out-of-scope, and assumptions – this is something that [..] sometimes we will ignore it, because it's not always relevant. So I guess I wouldn't want to have a low score because [..] this section is not perfect."* Participants did not want to be assessed negatively if using an element differently in their practice, or if not using it, pointing to the need for a customizable rubric.

The **inconsistent use of epics and master epics** came up as another challenge for measuring the quality of an epic based on a rubric. Some participants mentioned that some elements in our rubric, such as product outcome and instrumentation or assumptions, are defined at a higher level in master epics or product strategy documents; other participants only use master epics as the structure to represent an epic. Dependencies of elements across the two constructs should be taken into consideration when evaluating an epic or master epic so that product managers do not get penalized for missing elements.

Several participants mentioned the importance of **edge or boundary cases** in user stories and out-of-scope, as explained by P4 *"clarify[s] that this needs to be implemented or this doesn't need to be implemented or this needs to be tested or this doesn't need to be tested"*. Our rubric did not specifically emphasize those cases, and evaluations and recommendations for them will require a deeper semantic understanding of the feature and its domain versus simply testing attributes such as clarity and conciseness.

Finally, several participants pointed out that our rubric was **lacking guidance on sizing**, an important factor because epics should not be too small in scope, typically larger than what can get accomplished in a sprint, but also not too large. This scope of an epic is typically reflected in user stories and requirements but the number of stories and requirements are not a good predictor for the scope necessarily. Proper sizing requires experience and deep product knowledge, which could pose a challenge for an LLM-based evaluation.

4.2.3 Summary. For RQ2, our findings indicate that product managers are mostly satisfied with the rubric we developed but pointed out several pain points that need to be addressed, such as missing elements, rigidity, testing for edge and boundary cases, lack of sizing, or getting penalized for missing elements because of the structural breakdown into master epics and epics. The rubric is comprehensive and covers many of the quality criteria participants mentioned to us before showing them the rubric; however, evaluating semantic quality aspects of an epic (strategy alignment, sizing, customer needs) requires product or domain knowledge which poses a challenge for LLMs. It is also interesting to note that "completeness" was mentioned as a quality criteria while at the same time, participants did not want to get penalized for missing elements. Consistent with our findings in RQ1, this suggests that LLM evaluation needs to be stage-aware and customizable to stage and team-specific definitions of "complete."

4.3 RQ3: How viable is generative AI for supporting product managers in evaluating epics?

4.3.1 Current uses and perceptions of AI. The majority of the participants reported using generative AI in some capacity in their daily work (15/17). Uses of generative AI among participants mostly included some form of **content generation and summarisation**. In the context of agile epics, it is mostly used during discovery for **market research** and during epic creation to write **requirements, user stories, and test scenarios**.

Nearly half of the participants used an internal experimental tool for this purpose, as described by P12: *"Once I've kind of got a structure and subject matter content together, then I use AI for writing it, paraphrasing it, condensing it, summarising it, et cetera."* The 2 participants who were not using AI cited either a **lack of need or a lack of awareness of suitable internal tools** as the primary reason.

When asked about their confidence in AI-generated output, most participants expressed a high level of **trust for low-risk, simple and structured tasks**, viewing AI-generated content as a valuable starting point. P6 noted, *"I mean it's all relatively low risk, things that I'm doing. It's not like I'm having it interact with any systems, and ultimately I can edit the content as much as I want before I do anything else with it. So I feel totally fine using it now."* However, nearly all participants emphasized the importance of a **human-in-the-loop** approach, where they retain control to edit and refine the AI output. For example, P5 shared their envisioned use of the tool: *"we take what the AI tools generate as a starter and add our own details and elements to it."*

4.3.2 Values and desired uses of LLM evaluations. In reviewing the Epic Evaluator's assessment of their sample epics, 15/17 participants agreed with aspects of the score or recommendations. Similar to their positive sentiments on the rubric, some participants expressed enthusiasm for adopting a tool like Epic Evaluator: P14 said the evaluation is *"really good [..] I really want to use it right now,"* P2 compared it to *"my peer giving me feedback,"* and P8 said, *"I'm very excited about this [..] I think this will be a great tool."*

Participants' enthusiasm stemmed in part from identifying several specific uses in which LLM evaluations can provide value, indicating that generative AI can be viable in supporting epic evaluation given sufficient oversight and control over outcomes. Many participants indicated that they would use an LLM-based evaluation in their work to **augment stakeholder and peer feedback** (11/17) – as P15 explained, an LLM evaluation is *"not in lieu of an experienced product manager and getting their feedback, but it gives them another tool to build strong [..] epics."* Participants envisioned using LLM evaluations as a **starting point**, to drive **general improvements**, to get **accelerated feedback** or to evaluate and improve a **specific aspect of epic quality**, particularly those described in Section 4.2.1. For example, P5 wanted to assess clarity: *"I would use it to make sure [..] we provide everything clear and crisp for the development and design to take place."* Leveraging the strengths of automated

evaluation, some participants felt that the tool would be useful for **bulk evaluations** of epics across their team and identifying opportunities for improvement. P8, a team manager, commented on the difficulty of managing large quantities of epics within their team and the potential for LLM support: *"We have a lot of them [..] at least two pages worth of master epics in here. So if I could, for example, [..] be like, analyze all the master epics in this report, give me feedback, [..] I would definitely try that."* Other team managers also envisioned **team usage** of LLM evaluations; some specifically felt that it could act as a **training tool** to help novice product managers write higher quality epics. However, some managers also specified that they would not use this tool to evaluate employee performance, and one non-manager similarly expressed apprehension: *"[if we're] doing the work just for the sake of the score, I'll be starting to worry - isn't my manager being reported on the score?"*

4.3.3 Needs and barriers for LLM evaluation. To understand *how* an LLM should provide feedback, we reviewed participants' responses on their preferred rating systems. Participants shared mixed opinions on the idea of receiving ratings and how they should be given, but largely agreed that **clear, actionable recommendations should be the focus**, as this type of feedback would help them improve their epics. As P2 explained, *"verbal feedback is more actionable than scores,"* and *"for me, the important thing will be, what do I need to improve."* Some expressed reservations about scoring in general, including the **fear of being ranked, uncertainty and inconsistencies** in score interpretations, and the potential for scores to distract from the primary goal of improving epic quality. For example, P5 said, *"if we introduce numbers, some people would always want to have a high score or, you know, 100 out of 100, and that might introduce iterations that might cause delays."* On the other hand, some felt that including a score to accompany recommendations was acceptable or even desirable for motivating them to make improvements. Among those who supported the idea of scoring, there was a preference for a multi-parameter approach. Suggestions included rating individual aspects of the epic, such as clarity and completeness, rather than assigning a single overall score. Many participants suggested using a **three-point scale (e.g., high-medium-low)** to avoid *"splitting hairs"* over more granular scales (7/17). However, a three-point scale may be less useful if the majority of scores land in the middle, as some of our participants experienced. P7 commented, *"I didn't get anything because both of my things were middle of the road, [..] who cares, you know? There's a lot of things that could be 5 out of 10. So I would have liked it to emphasize on one side of the spectrum."*

In discussing instances of disagreement with the LLM evaluation, participants also raised barriers to adoption and suggestions to improve the value provided by LLM evaluations. Several participants wanted more **specificity**, either to content in the epic or to their domain (6/17). P11 gave an example of desired specificity: *"especially for NFRs, which might tend to be a long list, is there a way to kind of say, these specific ones need work?"* Participants also wanted more **actionable** recommendations - ones that they can implement to meaningfully improve epic quality (5/17). In some cases, recommendations were less actionable due to **lack of feasibility**. As P9 explained, *"many of the things are dependent on information I have no access [..] so as much as I agree with the points, I would have not been able to change it much after reading the recommendations anyway."*

Among the 17 participants, 5 discussed the need for LLMs to have **domain and stakeholder knowledge** to improve specificity and actionability. P12 gave an example of how the LLM's lack of stakeholder awareness resulted in unhelpful recommendations: *"for my product line, when I talk in terms of post-purchase fulfillment organization as a keyword, my [stakeholders..], they all get it. [..] A tool that is untrained will not be able to make sense of it and will keep asking for more and more information."* P7 called out a similar instance in which the LLM incorrectly flagged their product name as jargon: *"[The stakeholders] are assigned to the product, they know what [product name] is, [..] so I don't know that I would classify the product name for my product as [..] jargon."* Some participants questioned the LLM's ability to assess the quality of epic elements without domain awareness. As P17 noted, *"what's hard for outcomes and instrumentation is the number to measure with. I don't know if the system knows that this thing is even measurable."*

Another major cause of poor feedback was the LLM's lack of consideration for variability in epic creation practices (5/17). Some participants were concerned being penalized for differing formats, as highlighted by P6, *"to me, you can have an acceptable user story that doesn't necessarily so rigidly follow the, 'as a user, I want [goal] so that [benefit]' format"*. Others use epic elements in unique ways that were not accounted for in the development of Epic Evaluator, such as P2, who said, *"the way we usually use the nonfunctional requirements is for me, as a PM, [..] I am the only one who will read what I write in this section, [..] so this is why the feedback here, it's not very actionable. Is it important that it's not clear if it's something that I am writing for myself? Probably not."* Others were concerned about being penalized for missing elements, either during earlier stages of creation or for elements not applicable to their team. In these cases, they suggested focusing on evaluating individual elements rather than the overall epic. Participants also noted that epics can vary in type and complexity: as P8 recommended to *"first, categorize the type of epic and then depending on the type of epic, apply a specific rating scale to it."* Due to the Epic Evaluator's current limitation in evaluating diverse epics, elements, and formats, P2 said, *"the concern would be about [..] ranking epics based on this because I guess every team has like different methodologies."* Hence, participants felt that LLM feedback should **accommodate variability in practices** and avoid penalties for discrepancies.

Given these limitations, participants emphasized the need for human-in-the-loop when using LLM-based epic evaluations, similar to their approach to existing AI tools (see 4.3.1).

As P12 explained, *"I'm not able to trust any assistant completely. I will want to review it. I will read a recommendation it gives back to me and apply my own judgement to decide whether I can trust it completely."* They felt that they could trust the evaluation to some degree due to its similarity to human feedback and its usefulness – P10 felt that *"the recommendation is really good [..] and again, it's up to me if I find it not applicable, maybe I will not apply the recommendation, but it is still very helpful."* On the other hand, participants also recognized limitations in LLMs' contextual knowledge that lowered their trust. For example, P17 questioned, *"is it really giving me the right recommendation? [..] It's not trained on my product and my data, so it doesn't really know that I have another epic that's*

doing something else related to this." Trust was also lower when participants themselves lacked knowledge of the domain and hence could not properly assess the quality of LLM outputs. Participants' ratings of trust in the LLM evaluation reflected these mixed sentiments. Among the participants who rated their trust on a scale of 1-5, with 5 indicating highest trust, the average rating was 3.5. As P17 summarized, *"I think complete trust would mean it makes blind changes without my involvement, and it's not that."*

Finally, participants discussed the importance of **integration into their existing workflows** to ensure LLM evaluations do not create additional overhead. P6 cited this consideration as a major potential barrier to adoption: *"the friction of going into Aha, copying [the epic], going into the chat, pasting it like that back and forth, that is probably the main element that would stop me."* Similarly, P14 said, *"meeting me in the tools where I'm working is probably the biggest requirement for me."*

4.3.4 Summary. To address RQ3, we found that most of the product managers we interviewed were already using generative AI in their day-to-day work which is a great basis for the adoption of LLM evaluations. Participants were enthusiastic about Epic Evaluator and saw value in specific uses such as augmenting stakeholder and peer feedback, general improvements, accelerated feedback, bulk evaluations for managers, and as a training tool for novices. We observed additional needs and potential barriers to adoption including lack of specificity, need for actionable recommendations, lack of domain and stakeholder knowledge, importance of integration into their existing workflows, and lack of consideration for variability in their practices (consistent with findings in RQ2). Ratings seemed less important as compared to actionable recommendations, and a three-point scale appeared to be sufficient for most. In the following section, we share trade-off considerations and recommendations to improve the value of LLM evaluations.

5 Discussion and Implications

This work sought to investigate the use of AI evaluations in a new application area: agile epics. Agile epics are a critical artifact in the software development life cycle as they communicate detailed requirements to design and development. Creating high quality epics is a time-consuming task for product managers. Nonetheless, many of our participants established practices to improve the quality of epics for themselves or for their teams through peer reviews, manual scoring systems used for training new product managers, developing their own quality standards for their teams, and welcoming more standardization across the company. Hence, leveraging generative AI to help with quality assurance was an exciting prospect for our participants. As such, LLM evaluations can help time-pressured product managers craft higher quality epics, alleviating downstream problems such as churn, communication breakdowns, product delays, and cost overruns.

Our findings indicate that LLM evaluations of epics are viable and can provide value today. Participants were most appreciative of the explanations and recommendations that contained concrete examples from the epic. They were less interested in getting rated but rather being advised on how to improve their epics more efficiently. As such, they expressed strong interest in actionable suggestions, which blurs the boundaries between an epic evaluation and authoring tool. Managers saw value in knowing which epics from their team need their attention and an easy way to spot them among a larger body of epics.

5.1 LLM evaluations should be augmented with domain-specific knowledge

While explanations and recommendations were often grounded in the epic being evaluated, sometimes they didn't make sense because of the lack of domain or context knowledge. Domain knowledge includes, for example, product information, product-specific terms, related epics, overall objectives from master epics, and stakeholder information. The lack of domain knowledge has implications for properly measuring the "semantic" qualities of epics but also sometimes leads to a lack of specificity or actionable guidance.

This problem can be addressed through both human-driven and automated approaches. For example, product managers could augment LLM evaluations with domain-specific knowledge relevant to the evaluation task. However, asking product managers to manually add this knowledge would introduce additional barriers. A retrieval augmented generation (RAG) approach may be more feasible [40]. However, relevant content would need to be curated and stored in a knowledge base. As much of the background research for a new product often happens during the discovery phase, we believe an interesting approach would be to make that knowledge available through vector databases or search indices [51, 81] so that it can be accessed through a RAG system during epic evaluations.

5.2 Ensure ratings are clear and actionable

While ratings were perceived as less important, they remain necessary for drawing attention to parts that require improvement, particularly for managers who wanted to use LLMs for bulk evaluations. Their lower perceived importance could be due to the scores participants received in the study. As some of our participants observed, the medium rating was most prevalent in the samples we evaluated. This may indicate an issue with the definition of our rating scale, and it validates previously identified challenges that LLM evaluators often face in deciding between multiple options, which can be measured through positional bias [4, 37]. Findings are inconclusive on what the best approach is, but they point to users' needs for clear, distinct ratings as a characteristic of actionable LLM evaluations. To address these needs, we aim to further experiment with rating scales that have more differentiating definitions or point systems that can more clearly correlate between high and low epic quality. Inspired by some more recent LLM-as-a-Judge approaches [74], a Chain-of-Thought approach [77] could be used by first asking an LLM to recommend improvements or produce a general assessment of an element, then feeding this output into a subsequent evaluation step with a rating scale defined based on the types of recommendations.

5.3 LLM evaluations should be stage-aware

When we discussed the frequency of evaluation with our participants, a pattern for injecting LLM evaluations into their work practices evolved. While one asked for real-time evaluations, the majority seemed to converge around four stages in the creation

process: (1) after finishing an initial draft, (2) after refining the draft with other product managers on the team, and before sharing with design and development, (3) after discussions with design and development leads, and (4) before delivering to design and development. However, our interviews revealed that participants were concerned about an evaluation tool that gives non-favorable or less useful ratings to parts of an epic that are not well developed yet because, as many of them emphasized, an epic goes through stages. An epic often starts with title, problem statement and user stories, and other elements are added and evolve over time in a collaborative process. While many mentioned completeness as an important quality criteria, epics in earlier stages are not necessarily low quality because they are not complete. An evaluation tool should be aware of the current stage of an epic, provide an option for users to indicate the stage, or allow users to select which elements to evaluate at a given time. Stage-aware evaluations also have implications for generating an overall epic rating, since elements that are not relevant at a certain stage should not be included in the overall score.

5.4 LLM support should be tightly integrated into existing agile management tools

While many participants reacted positively about using an LLM evaluation tool, they indicated that it needs to be tightly integrated into their existing epic management applications so it's easy-to-use on demand. While our experimental Epic Evaluator tool provides simple way to evaluate epics through pasting the text of the full epic into a text box, it requires users to extract the elements of an epic beforehand. This poses a challenge given the diversity of tools being used in our company and the somewhat limited export capabilities. We experienced this first-hand when we converted epics that participants shared with us to evaluate for our concept testing. For tooling to be adopted, there needs to be either an integration at the API level or it needs to be integrated as a plugin into existing agile management tools. For future versions of our tool, we are currently exploring various design integrations.

5.5 LLM evaluations should drive consistency in epics while providing flexibility and user control to support diverse practices

We were surprised by the diversity of existing epic creation practices across our organization despite the existence of a standard template. We learned that different teams had created their own templates based on or inspired by the standard template, while other teams had developed their own methodology from the ground up or used approaches that came in through company acquisitions. Despite diversity in practices, the standard elements we used for our rubric seemed familiar to most, and more standardization was viewed as acceptable and even desirable. There mostly seemed to be an agreement that problem statement, and user stories (including requirements) are the most important elements of an epic. Importance seemed less clear with other elements. Overall, the rubric and the quality criteria we had developed seemed valuable to participants, but the rubric was missing some elements they used in their practices, and we received feedback on how to improve the quality criteria in various ways. While some improvements may require more research, such as the addition of domain-specific knowledge as outlined above, other changes will be easily incorporated into the next version of the rubric. In order to support the need and desire for standardization while preserving some flexibility, we propose the creation of a core template and extensions that support different practices. The rubric would be aligned and customizable to the teams' needs at different stages, and an evaluation tool should be built on a custom rubric.

5.6 Beyond LLM support

Ultimately, we recognize that LLMs are only one piece of the puzzle in addressing product managers' needs, with the potential to augment parts of their work but not to automate it entirely. One way to look at agile epics is that they serve as an instrument to translate findings from the discovery phase into actionable requirements for the delivery phase of a new product or feature through a highly collaborative, iterative process. While our findings indicate that LLM evaluations of epics can improve aspects of this process, a holistic approach is needed to fully address communication breakdowns from discovery to delivery. Participants pointed out other causes of breakdowns, such as the usage of diverse and disconnected tools and limited stakeholder access to these tools. While our standard template contains a section on questions and decisions meant to encourage stakeholder discussions around an epic, those discussions, in practice, happen mostly outside the main agile management tool using Slack, email, file sharing, or other online discussion forums. In addition to leveraging LLM evaluation tools, we also suggest practitioners review their existing agile project management tooling portfolio to further reduce communication breakdowns. As such, product managers remain indispensable in the complex work of managing epics and cross-functional coordination.

6 Limitations

As an industry case study, our research was conducted in our own company, potentially limiting our findings. First, we were able to experiment only with LLMs that were approved for safe use within IBM. This excluded the use of closed source models such as, GPT-4 in our model evaluation – those models may outperform the models we used. Second, while we have iterated over our prompts, we acknowledge that prompts are known to be sensitive to even small changes and changes of the LLM being used and may not yield the same performance under different conditions [56]. Third, although agile practices have similarities across many companies, they may not be the same. We acknowledge that our research may not be readily transferrable to other organizations because of IBM-specific agile practices, for example, related to elements of an epic being used, epic creation process and stakeholders, or quality expectations. Last, since we concept-tested LLM output through screenshots, our participants did not have the opportunity to interact directly with the tool we created. While this approach elicited valuable insight into the viability and barriers of our rubric and LLM-assisted epic evaluations, it may limit insights into real-world usage.

7 Conclusion

This industry case study explored the potential of using LLMs to evaluate the quality of agile epics, a core component of agile software development. Interviews with 17 product managers of

varying experience levels, during which we also concept-tested a rubric and LLM evaluations based on the rubric, highlighted both the opportunities and limitations of LLM evaluations in supporting the epic creation process in our company. Our participants were enthusiastic about using LLM evaluations to improve the quality of their epics or their team's epics, addressing problems of churn, communication breakdowns, product delays, and cost overruns. At the same time, our findings also underscore the need for refinement to make this approach more effective and adaptable to existing agile workflows. Our future work focuses on improving explanations and recommendations through the integration of domain knowledge as a fundamental challenge. We also plan to extend this research to incorporate views of other stakeholders, such as design and development, who are on the receiving end of an agile epic.

While this study was in the context of agile practices in our company, we believe that lessons learned from this work are meaningful for both the "reflective practitioner" [62] who seeks to infuse generative AI capabilities into agile work practices in their own organization and the researcher exploring areas of AI usage that have been traditionally very challenging as they require a lot of human decision-making, insight, and oversight. In practice, achieving performance improvements by integrating AI assistance into human workflows is challenging and may even lead to performance degradation [16]. AI assistance needs to be carefully designed to be successful, and we believe that our research has multiple contributions towards that goal, including a novel rubric that we validated, a set of prompts for quality evaluation, detailed insights into current agile practices, the role of LLM evaluations in quality evaluations, and validation that LLM evaluations are a viable approach to facilitate how agile software teams create and manage epics in the future to successfully deliver high-quality, user-centered software products.

Acknowledgments

We would like to express our sincere gratitude to the members of the agile software development leadership team and the AI productivity team at IBM for their invaluable support and collaboration throughout this project.

A special thanks to Ed Jacob-Moffatt, Gord Davison, Michael Desmond, Praveen Anthony, and Adam Chase for their guidance, constructive feedback, and continuous encouragement. Their expertise and commitment have significantly contributed to the development and refinement of our ideas.

References

[1] M. O. Ahmad, J. Markkula, M. Oivo, and P. Kuvaja. 2018. Kanban in software development: A systematic literature review. *Journal of Systems and Software* 137 (2018), 96–113. doi:10.1016/j.jss.2017.11.049

[2] Albertus M. M. Antaputra and Lim Tek Yong. 2024. A Systematic Literature Review on Agile Requirements Engineering and Artificial Intelligence. In *2024 22nd International Conference on ICT and Knowledge Engineering (ICT&KE)*. 1–6. doi:10.1109/ICTKE62841.2024.10787176

[3] Chetan Arora, John Grundy, and Mohamed Abdelrazek. 2024. *Advancing Requirements Engineering Through Generative AI: Assessing the Role of LLMs*. Springer Nature Switzerland, Cham, 129–148. doi:10.1007/978-3-031-55642-5_6

[4] Zahra Ashktorab, Michael Desmond, Qian Pan, James M. Johnson, Martin Santillan Cooper, Elizabeth M. Daly, Rahul Nair, Tejaswini Pedapati, Swapnaja Achintalwar, and Werner Geyer. 2024. Aligning Human and LLM Judgments: Insights from EvalAssist on Task-Specific Evaluations and AI-assisted Assessment Strategy Preferences. arXiv:2410.00873 [cs.HC] https://arxiv.org/abs/2410.00873

[5] Atlassian. 2025. Epics in Agile Project Management. https://www.atlassian.com/agile/project-management/epics Accessed: 2025-02-02.

[6] Anas BAHI, Jihane GHARIB, and Youssef GAHI. 2024. Integrating Generative AI for Advancing Agile Software Development and Mitigating Project Management Challenges. *International Journal of Advanced Computer Science and Applications* 15, 3 (2024). doi:10.14569/IJACSA.2024.0150306

[7] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. 2001. Manifesto for Agile Software Development. https://agilemanifesto.org/. Accessed: 2025-01-24.

[8] Woubshet Behutiye, Pilar Rodríguez, and Markku Oivo. 2022. Quality requirement documentation guidelines for agile software development. *IEEE Access* 10 (2022), 70154–70173.

[9] Woubshet Behutiye, Pertti Seppänen, Pilar Rodríguez, and Markku Oivo. 2020. Documentation of quality requirements in agile software development. In *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering*. 250–259.

[10] Niels Bik, Garm Lucassen, and Sjaak Brinkkemper. 2017. A reference method for user story requirements in agile systems development. In *2017 IEEE 25th international requirements engineering conference workshops (REW)*. IEEE, 292–298.

[11] B. Boehm and R. Turner. 2004. *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley.

[12] Michelle Brachman, Amina El-Ashry, Casey Dugan, and Werner Geyer. 2024. How Knowledge Workers Use and Want to Use LLMs in an Enterprise Context. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI EA '24)*. Association for Computing Machinery, New York, NY, USA, Article 189, 8 pages. doi:10.1145/3613905.3650841

[13] Jill Burstein. 2003. The E-rater® Scoring Engine: Automated Essay Scoring with Natural Language Processing. In *Automated Essay Scoring: A Cross-disciplinary Perspective*, Mark D. Shermis and Jill Burstein (Eds.). Lawrence Erlbaum Associates, Inc., 113–121.

[14] Birgitta Böckeler, Adhavan KP, Sayantan Mukhopadhyay, Radhika Sivadass, and Jaiganesh B. 2024. Using AI for requirements analysis: A case study. https://www.thoughtworks.com/en-us/insights/blog/generative-ai/using-ai-requirements-analysis-case-study. Accessed: 2025-01-10.

[15] Alexia Cambon, Brent Hecht, Ben Edelman, Donald Ngwe, Sonia Jaffe, Amy Heger, Mihaela Vorvoreanu, Sida Peng, Jake Hofman, Alex Farach, et al. 2023. Early LLM-based Tools for Enterprise Information Workers Likely Provide Meaningful Boosts to Productivity. *Microsoft Research. MSR-TR-2023-43* (2023).

[16] Andres Campero, Michelle Vaccaro, Jaeyoon Song, Haoran Wen, Abdullah Almaatouq, and Thomas W. Malone. 2022. A Test for Evaluating Performance in Human-Computer Systems. arXiv:2206.12390 [cs.HC] https://arxiv.org/abs/2206.12390

[17] Peter W Cardon, Kristen Getchell, Stephen Carradini, Carolin Fleischmann, and James Stapp. 2023. Generative AI in the workplace: Employee perspectives of ChatGPT benefits and organizational policies. (2023).

[18] Haowei Cheng, Jati H Husen, Sien Reeve Peralta, Bowen Jiang, Nobukazu Yoshioka, Naoyasu Ubayashi, and Hironori Washizaki. 2024. Generative AI for Requirements Engineering: A Systematic Literature Review. *arXiv preprint arXiv:2409.06741* (2024).

[19] Cheng-Han Chiang, Wei-Chih Chen, Chun-Yi Kuan, Chienchou Yang, and Hung-yi Lee. 2024. Large language model as an assignment evaluator: Insights, feedback, and challenges in a 1000+ student course. *arXiv preprint arXiv:2407.05216* (2024).

[20] Cheng-Han Chiang and Hung-yi Lee. 2023. Can large language models be an alternative to human evaluations? *arXiv preprint arXiv:2305.01937* (2023).

[21] Victoria Clarke and Virginia Braun and. 2017. Thematic analysis. *The Journal of Positive Psychology* 12, 3 (2017), 297–298. doi:10.1080/17439760.2016.1262613 arXiv:https://doi.org/10.1080/17439760.2016.1262613

[22] Fabiano Dalpiaz and Sjaak Brinkkemper. 2021. Agile Requirements Engineering: From User Stories to Software Architectures. In *2021 IEEE 29th International Requirements Engineering Conference (RE)*. 504–505. doi:10.1109/RE51729.2021.00076

[23] Hoa Khanh Dam, Truyen Tran, John Grundy, Aditya Ghose, and Yasutaka Kamei. 2019. Towards effective AI-powered agile project management. In *2019 IEEE/ACM 41st international conference on software engineering: new ideas and emerging results (ICSE-NIER)*. IEEE, 41–44.

[24] Michael Desmond, Zahra Ashktorab, Qian Pan, Casey Dugan, and James M Johnson. 2024. EvaluLLM: LLM assisted evaluation of generative outputs. In *Companion Proceedings of the 29th International Conference on Intelligent User Interfaces*. 30–32.

[25] T. Dingsøyr, S. Nerur, V. Balijepally, and N. B. Moe. 2012. A decade of agile methodologies: Towards explaining agile software development. *Journal of Systems and Software* 85, 6 (2012), 1213–1221. doi:10.1016/j.jss.2012.02.033

[26] Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy S Liang, and Tatsunori B Hashimoto. 2024. Alpacafarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems* 36 (2024).

[27] António MS Ferreira, Alberto Rodrigues da Silva, and Ana CR Paiva. 2022. Towards the Art of Writing Agile Requirements with User Stories, Acceptance Criteria, and Related Constructs.. In *ENASE*. 477–484.

[28] International Organization for Standardization. 2018. ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering. 104 pages.

[29] Jie Gao, Simret Araya Gebreegziabher, Kenny Tsu Wei Choo, Toby Jia-Jun Li, Simon Tangi Perrault, and Thomas W. Malone. 2024. A Taxonomy for Human-LLM Interaction Modes: An Initial Exploration. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–11.

[30] Geeks For Geeks. 2025. Epics. https://www.geeksforgeeks.org/what-are-epics-user-stories-in-agile/ Accessed: 2025-02-05.

[31] Gradio Team. 2025. Gradio: Build Machine Learning Apps Quickly and Easily. https://www.gradio.app/ Accessed: 2025-01-24.

[32] A. Hussain, E. O. Mkpojiogu, and F. M. Kamal. 2016. The role of requirements in the success or failure of software projects. *International Review of Management and Marketing* 6, 7 (2016), 306–311.

[33] Bill Iuso. 1975. Concept testing: an appropriate approach. *Journal of Marketing research* 12, 2 (1975), 228–231.

[34] M. I. Kamata and T. Tamai. 2007. How does requirements quality relate to project success or failure?. In *15th IEEE International Requirements Engineering Conference (RE 2007)*. IEEE, 69–78. doi:10.1109/RE.2007.45

[35] Rashidah Kasauli, Eric Knauss, Jennifer Horkoff, Grischa Liebel, and Francisco Gomes de Oliveira Neto. 2021. Requirements engineering challenges and practices in large-scale agile system development. *Journal of Systems and Software* 172 (2021), 110851.

[36] Rashidah Kasauli, Grischa Liebel, Eric Knauss, Swathi Gopakumar, and Benjamin Kanagwa. 2017. Requirements Engineering Challenges in Large-Scale Agile System Development. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*. 352–361. doi:10.1109/RE.2017.60

[37] Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, et al. 2023. Prometheus: Inducing fine-grained evaluation capability in language models. In *The Twelfth International Conference on Learning Representations*.

[38] Tae Soo Kim, Yoonjoo Lee, Jamin Shin, Young-Ho Kim, and Juho Kim. 2024. Evallm: Interactive evaluation of large language model prompts on user-defined criteria. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 1–21.

[39] Abhishek Kumar, Sonia Haiduc, Partha Pratim Das, and Partha Pratim Chakrabarti. 2024. LLMs as Evaluators: A Novel Approach to Evaluate Bug Report Summarization. *arXiv preprint arXiv:2409.00630* (2024).

[40] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[41] Sebastian Lubos, Alexander Felfernig, Thi Ngoc Trang Tran, Damian Garber, Merfat El Mansi, Seda Polat Erdeniz, and Viet-Man Le. 2024. Leveraging llms for the quality assurance of software requirements. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 389–397.

[42] Simone Luchini, Nadine T Maliakkal, Paul V DiStefano, John D Patterson, Roger Beaty, and Roni Reiter-Palmon. 2023. Automatic Scoring of Creative Problem-Solving with Large Language Models: A Comparison of Originality and Quality Ratings. (2023).

[43] Shengjie Ma, Chong Chen, Qi Chu, and Jiaxin Mao. 2024. Leveraging Large Language Models for Relevance Judgments in Legal Case Retrieval. *arXiv preprint arXiv:2403.18405* (2024).

[44] Julia M. Markel, Steven G. Opferman, James A. Landay, and Chris Piech. 2023. GPTeach: Interactive TA Training with GPT-based Students. In *Proceedings of the Tenth ACM Conference on Learning @ Scale* (Copenhagen, Denmark) *(L@S '23)*. Association for Computing Machinery, New York, NY, USA, 226–236. doi:10.1145/3573051.3593393

[45] Nuno Marques, Rodrigo Rocha Silva, and Jorge Bernardino. 2024. Using ChatGPT in Software Requirements Engineering: A Comprehensive Review. *Future Internet* 16, 6 (2024), 180.

[46] Mio Mattila. 2023. Assisting product line thinking and information sharing using a real-time visual tracking model for agile epics. (2023).

[47] Thiago Souto Mendes, Mário André de F. Farias, Manoel Mendonça, Henrique Frota Soares, Marcos Kalinowski, and Rodrigo Oliveira Spínola. 2016. Impacts of agile requirements documentation debt on software projects: a retrospective study. In *Proceedings of the 31st annual ACM symposium on applied computing*. 1290–1295.

[48] Shahraz Nasir, Eduardo Guerra, Luciana Zaina, and Jorge Melegati. 2023. An Exploratory Study About Non-functional Requirements Documentation Practices in Agile Teams. In *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*. 1009–1017.

[49] Sofia Nilsson and Emilia Palm. 2024. Optimization of Portfolio Management How to prioritize work within different departments and portfolios. (2024).

[50] Johannes J Norheim, Eric Rebentisch, Dekai Xiao, Lorenz Draeger, Alain Kerbrat, and Olivier L de Weck. 2024. Challenges in applying large language models to requirements engineering tasks. *Design Science* 10 (2024), e16.

[51] Elastic N.V. 2025. Elasticsearch - The Distributed Search and Analytics Engine. https://www.elastic.co/elasticsearch/. Accessed: 2025-02-07.

[52] Johanna Olander and Johanna Qvist. 2023. Agile Planning Activities and Team Characteristics for On-time Delivery in Software Development Teams: A case study at Ericsson.

[53] Jay U. Oswal, Harshil T. Kanakia, and Devvrat Suktel. 2024. Transforming Software Requirements into User Stories with GPT-3.5 -: An AI-Powered Approach. In *2024 2nd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*. 913–920. doi:10.1109/IDCIoT59759.2024.10467750

[54] Qian Pan, Zahra Ashktorab, Michael Desmond, Martin Santillan Cooper, James Johnson, Rahul Nair, Elizabeth Daly, and Werner Geyer. 2024. Human-Centered Design Recommendations for LLM-as-a-judge. *arXiv preprint arXiv:2407.03479* (2024).

[55] Krishna Ronanki, Christian Berger, and Jennifer Horkoff. 2023. Investigating ChatGPT's potential to assist in requirements elicitation processes. In *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 354–361.

[56] Paul Röttger, Valentin Hofmann, Valentina Pyatkin, Musashi Hinck, Hannah Kirk, Hinrich Schuetze, and Dirk Hovy. 2024. Political Compass or Spinning Arrow? Towards More Meaningful Evaluations for Values and Opinions in Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15295–15311. doi:10.18653/v1/2024.acl-long.816

[57] W. W. Royce. 1970. Managing the development of large software systems. In *Proceedings of IEEE WESCON*. 1–9.

[58] Cheol Ryu, Seolhwa Lee, Subeen Pang, Chanyeol Choi, Hojun Choi, Myeonggee Min, and Jy-Yong Sohn. 2023. Retrieval-based Evaluation for LLMs: A Case Study in Korean Legal QA. In *Proceedings of the Natural Legal Language Processing Workshop 2023*. 132–137.

[59] SAFe. 2025. NFRs. https://scaledagileframework.com/nonfunctional-requirements Accessed: 2025-02-05.

[60] SAFe. 2025. User Stories. https://businessmap.io/agile/project-management/user-stories Accessed: 2025-02-05.

[61] Malik Abdul Sami, Muhammad Waseem, Zheying Zhang, Zeeshan Rasheed, Kari Systä, and Pekka Abrahamsson. 2024. AI based Multiagent Approach for Requirements Elicitation and Analysis. *arXiv preprint arXiv:2409.00038* (2024).

[62] D Schon. 1983. *The reflective practitioner. How professionals think in action*. Temple Smith, London.

[63] K. Schwaber and J. Sutherland. 2020. *The Scrum Guide*. Scrum Alliance. https://scrumguides.org/

[64] Scrum Alliance. 2025. Understanding Epics. https://resources.scrumalliance.org/Article/epic-agile Accessed: 2025-02-05.

[65] Orit Shaer, Angelora Cooper, Osnat Mokryn, Andrew L Kun, and Hagit Ben Shoshan. 2024. AI-Augmented Brainwriting: Investigating the use of LLMs in group ideation. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 1050, 17 pages. doi:10.1145/3613904.3642414

[66] Vaishali Siddeshwar, Sanaa Alwidian, and Masoud Makrehchi. 2024. A Systematic Review of AI-Enabled Frameworks in Requirements Elicitation. *IEEE Access* 12 (2024), 154310–154336. doi:10.1109/ACCESS.2024.3475293

[67] Henrique F Soares, Nicolli SR Alves, Thiago S Mendes, Manoel Mendonça, and Rodrigo O Spinola. 2015. Investigating the link between user stories and documentation debt on software projects. In *2015 12th International Conference on Information Technology-New Generations*. IEEE, 385–390.

[68] Yishen Song, Qianta Zhu, Huaibo Wang, and Qinhua Zheng. 2024. Automated Essay Scoring and Revising Based on Open-Source Large Language Models. *IEEE Transactions on Learning Technologies* (2024).

[69] Annalisa Szymanski, Noah Ziems, Heather A Eicher-Miller, Toby Jia-Jun Li, Meng Jiang, and Ronald A Metoyer. 2024. Limitations of the LLM-as-a-Judge Approach for Evaluating LLM Outputs in Expert Knowledge Tasks. *arXiv preprint arXiv:2410.20266* (2024).

[70] Tech Target. 2025. Agile Software Development. https://www.techtarget.com/searchsoftwarequality/definition/agile-software-development Accessed: 2025-02-05.

[71] Muhammad Aminu Umar and Kevin Lano. 2023. Automated Requirements Engineering in Agile Development: A Practitioners Survey. In *2023 3rd International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*. 1–7. doi:10.1109/ICECCME57830.2023.10253030

[72] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram Rajamani. 2023. Frustrated with code quality issues? llms can help! *arXiv preprint arXiv:2309.12938* (2023).

[73] Kelly B Wagman, Matthew T Dearing, and Marshini Chetty. 2025. Generative AI Uses and Risks for Knowledge Workers in a Science Organization. *arXiv preprint*

arXiv:2501.16577 (2025).

[74] Nico Wagner, Michael Desmond, Rahul Nair, Zahra Ashktorab, Elizabeth M. Daly, Qian Pan, Martín Santillán Cooper, James M. Johnson, and Werner Geyer. 2024. Black-box Uncertainty Quantification Method for LLM-as-a-Judge. arXiv:2410.11594 [cs.LG] https://arxiv.org/abs/2410.11594

[75] Chen Wang, Jin Zhao, and Jiaqi Gong. 2024. A Survey on Large Language Models from Concept to Implementation. arXiv:2403.18969 [cs.CL] https://arxiv.org/abs/2403.18969

[76] Z Wang, J Li, G Li, and Z Jin. 2023. ChatCoder: Chat-based Refine Requirement Improves LLMs' Code Generation. arXiv 2023. *arXiv preprint arXiv:2311.00272* (2023).

[77] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. *arXiv preprint* arXiv:2201.11903 (2022). https://arxiv.org/abs/2201.11903

[78] Young Urban Project. 2025. Epic in Agile. https://www.youngurbanproject.com/what-is-an-epic-in-agile/ Accessed: 2025-02-05.

[79] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2024. A Survey of Large Language Models. arXiv:2303.18223 [cs.CL] https://arxiv.org/abs/2303.18223

[80] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems* 36 (2023), 46595–46623.

[81] Zilliz. 2025. Milvus: Open-source Vector Database for Scalable AI Applications. https://milvus.io/. Accessed: 2025-02-07.

A Appendix

A.1 Rubric

Table 3: The rubric developed with subject matter experts. The rubric includes eight elements. For each element, we provide a definition and criteria as well as an example.

Element	Definition
Title	The title of an agile epic should be: • Short and concise (no more than 10 words), • Descriptive (capture the essence of the epic), and • Convey the primary objective of the epic (value of doing the epic). Example: "Enable Password Reset for Registered Users to Regain Account Access."
Problem Statement	The problem statement briefly summarizes the problem which is being solved in this epic. It also includes a brief description of the feature developed that addresses the problem, and the value for the customer or user and why they should be interested in adopting this new feature. The problem statement should be: • Clear: Free of jargon and ambiguity, the description should be easy for all team members and stakeholders to understand. • Comprehensive: The problem statement needs to contain three elements: (1) A description of the problem being solved, (2) a high level description of the feature addressing , and (3) the value for the customer. • Concise: It should be written without unnecessary detail or redundancy. Example: "Users struggle to regain access to their accounts when they forget their passwords. The current process is difficult and customers are abandoning the product or require help from support agents. This epic will implement a password reset feature, integrated with existing user management systems, to enable users to regain access to their account. This feature will reduce customer support requests for password recovery by 30%, improving user retention and reducing operational costs."
Product Outcome & Instrumentation	The "product outcome and instrumentations" element describes the overall goals of this feature. Product outcomes are what determines this feature's success. Good product goals should measure things like adoption, feature retention, satisfaction, engagement or impressions, general usage, completion percentages and speed. Bad product goals are measurements around lagging metrics like churn, revenue, customer health, renewal, etc. Instrumentation refers to metrics that can be used to measure the goals, e.g. click-throughs etc. Example: "Outcome: Reduce time to recover access by 50%, or increase number of users that can recover access to their account without help from support, Metric: Reduced number of support tickets on this topic"
Requirements - User Stories	User stories describe what the user needs or expects from the system. User stories are a fundamental part of an agile epic. They should be: • Clear: They should be written in a clear and user-centric way to communicate requirements. • Structured; They should be written in a structured and consistent format to ensure clarity, understanding, and alignment across the team. • Complete: They should also be complete in terms of what is needed for the overall feature. If there are too many user stories, the epic may need to be broken down. The structure should be as follows: As a [user role], I want [goal] so that [reason/benefit]. The who [user role], what [goal], why [benefit] format avoids getting into the how (as that should not be part of the epic). User stories identify and breakdown needs for each user role. If necessary, they can summarize bigger needs and then break down them down into sub-details. These details define the epic / feature needs. Example: "As a registered user, I need to be able to reset my password so that I can regain access to my account if I forget it."

Element	Definition
Requirements - Requirements	The requirements element breaks down each user story into a requirement statement but typically leaves out the reason/benefit found in the user story. It is basically a breakdown of a user story into system capabilities that are required to implement the user story. Example for the above user story: "The registered user should be able to access a password reset function" "The registered user will need to be validated prior to executing the reset" "The registered user should be able to choose a new password"
Assumptions	Assumptions are the beliefs or statements that the team holds to be true at the start of the epic but which may not yet be fully verified. They often represent things that need to be confirmed or validated during the development process. Assumptions are typically based on what is known at the time but can evolve or be disproved as more information becomes available. • Technical Assumption: "The current API version will support the new feature without requiring significant changes." • Business Assumption: "Users will find the new filtering functionality useful and increase their product search efficiency." • User Behavior Assumption: "The majority of users will use this feature on mobile devices rather than desktop." Assumptions provide guardrails for the feature being developed. They should be: • Explicit: Assumptions should be clearly stated and not left to implicit understanding. • Testable: Where possible, assumptions should be validated through early testing or research. • Impact: Their impact should be explained when listed. Example: "We assume that the existing user database can handle additional password reset requests without performance issues. This will be validated during testing."
Non-Functional Requirements	Non-Functional requirements are requirements that are not centered around the user experience. Examples include how development needs to handle cookie management, or how long data should be held for, or what sort of rate limits should be expected. Non-functional requirements should be: • Measurable: Non-functional requirements should be quantifiable (e.g., performance, security, usability). • Testable: Non-functional requirements should have clear criteria for validation. • Realistic: The requirements should be achievable within the project's constraints and technology stack. • Relevant: Non-functional requirements should address critical aspects like performance, security, or scalability, and not add unnecessary overhead. Example: "The system should respond to password reset requests within 1 second for 95
Out-of-Scope	The out-of-scope section clearly states what will not be included in the feature, usually as a list of items. Out-of-scope items should be: • Clear: Out-of-scope items should be clearly and explicitly defined, leaving no ambiguity. Stakeholders and team members should understand exactly what is being excluded. • Relevant: The out-of-scope items should be directly related to the epic, so they define realistic boundaries. For example, listing "completely different features" (like reporting or analytics) might be irrelevant if the epic is about search functionality. • Specific: High-quality out-of-scope items should specify particular features, functionalities, or tasks that will not be included. Vague statements (e.g., "some advanced features") can lead to confusion. • Justifiable: There should be a clear rationale for excluding certain elements. It could be due to time constraints, resource limitations, or the need to prioritize other features first. Example: "Out of Scope: Two-factor authentication (2FA) will not be implemented in this release."

A.2 Rating Scale Used

```
"""
- HIGH: HIGH quality means almost all requirements in the
QUALITY STANDARDS FOR {element_name} elements are met.
The quality is very good and needs little to no improvements.
- MEDIUM: MEDIUM quality means some requirements in the
QUALITY STANDARDS FOR {element_name} elements are met.
The quality is generally good but can be improved.
- LOW: LOW quality means many of the requirements in the
QUALITY STANDARDS FOR {element_name} elements are not met.
The quality is bad and needs improvement.
"""
```

A.3 Example Evaluation Prompt for Title Element

```
"""
You are a product manager and subject matter expert in
agile epics. Your task is to assess the AGILE EPIC below.

AGILE EPIC
{epic}

For this ASSESSMENT, please rate the following element
of the agile epic: TITLE

In your ASSESSMENT rate the QUALITY of the TITLE on the
following scale:

{scale}

Please rovide a detailed EXPLANATION for your assessment
with examples where the quality suffers and make
RECOMMENDATIONS on how to improve the TITLE. Follow the
QUALITY STANDARDS FOR TITLE strictly.

QUALITY STANDARDS FOR TITLE elements:
The title of an agile epic should be:
- Short and concise (no more than 10 words),
- Descriptive (capture the essence of the epic), and
- Convey the primary objective of the epic  (value of
doing the epic).

Example: “Enable Password Reset for Registered Users to
Regain Account Access."

In your ASSESSMENT please respond ONLY with a valid JSON
(no other text) in the following format:

{
   "assessments": [
        {
             "element": "<ELEMENT>",
             "quality": "<QUALITY>",
             "explanation": "<EXPLANATION>"
             "recommendations": "<RECOMMENDATIONS>"
        }
   ]
}

ASSESSMENT:
"""
```